

NutriWarrior Security Sprint Report

1. Estado inicial

O projeto tinha o Core API nutricional com Cliente, dias, refeições, itens, alimentos, metas, resumo diário, Flyway V1-V7 e OpenAPI. Não havia Spring Security, JWT, BCrypt, usuários ou ownership; qualquer consumidor podia usar qualquer clienteld.

O working tree estava limpo antes da sprint e as migrations V1-V7 foram preservadas.

2. Arquivos criados

- `V8\_\_create\_usuario\_and\_client\_ownership.sql`.
- Entidade `Usuario`, enum `Role` e `UsuarioRepository`.
- DTOs de registro, login, autenticação, usuário e novo paciente.
- `AuthService`, `UsuarioService`, `ClienteAccessService` e `PacienteService`.
- `AuthController`, `PacienteController`, `SecurityConfig`, `JwtProperties` e `SecurityErrorHandler`.
- `SecurityIntegrationTest` e suporte de autenticação dos testes legados.

3. Arquivos modificados

- `build.gradle.kts`: Spring Security, Resource Server e Security Test.
- `Cliente` e `ClienteRepository`: vínculos com usuário paciente e nutricionista.
- Serviços de Cliente, DiaRegistro, Refeicao, ItemRefeicao, Meta e Resumo: ownership.
- `FoodItemController`: leitura autenticada e escrita para nutricionista.
- `OpenApiConfig`: esquema Bearer JWT.
- propriedades de JWT e README.

4. Migrations

A V8 cria usuario, email único, role, ativo, data de criação, cliente.usuario_id, cliente.nutricionista_id, constraints e índices. V1-V7 não foram alteradas. Os vínculos são nullable para preservar clientes antigos, que permanecem inacessíveis até serem associados a uma identidade.

5. Modelo Usuario

Usuario contém id, nome, email normalizado, senhaHash, Role, ativo e createdAt. Senhas são codificadas com BCrypt e nunca aparecem nos DTOs.

6. Fluxo de autenticação

POST /auth/register/nutricionista cria nutricionista. POST /auth/login valida BCrypt e emite access token. GET /me retorna usuário autenticado e clienteld quando aplicável. Não há refresh token nesta sprint.

7. Estrutura do JWT

O token usa Nimbus/Spring Security, com sub igual ao id do usuário, claim role, iat e exp. A duração vem de JWT_EXPIRATION_MINUTES.

8. Roles

NUTRICIONISTA pode criar/listar pacientes vinculados, ler e escrever FoodItem e acessar seus próprios clientes. PACIENTE pode acessar apenas seu Cliente e ler FoodItem.

9. Relação Nutricionista -> Paciente

POST /nutricionistas/me/pacientes cria Usuario PACIENTE e Cliente na mesma transação, ligando ambos ao nutricionista autenticado. GET /nutricionistas/me/pacientes lista somente os pacientes daquele nutricionista.

10. Regras de autorização

ClienteAccessService é usado pelos serviços de Cliente, DiaRegistro, Refeicao, ItemRefeicao, Meta e Resumo. Ele compara o usuário autenticado com o vínculo persistido, retornando 403 para outro paciente ou nutricionista.

11. Endpoints públicos

GET /health, /auth/**, /swagger-ui/**, /swagger-ui.html e /v3/api-docs/**.

12. Endpoints protegidos

Todos os demais endpoints. O catálogo /api/foods permite GET para ambos os roles e POST/PUT/DELETE somente para NUTRICIONISTA.

13. Swagger

O OpenAPI agora declara bearerAuth como esquema HTTP Bearer JWT e inclui o requisito de segurança.

14. Testes executados

Foram adicionados testes MockMvc para registro, email duplicado, BCrypt, login válido/inválido, /me, 401, token inválido, criação/listagem de pacientes, isolamento entre nutricionistas e pacientes, fluxo diário e roles de FoodItem. Os testes legados foram adaptados com usuário de teste persistido e @WithMockUser, sem desabilitar filtros.

15. Resultado final do build

O comando executado foi:

```
powershell
```

```
.\gradlew.bat clean test
```